Parallel Savings Fix Audit Report

Prepared by Cyfrin

Version 1.0

Lead Auditors

Stalin

Alix40

T1MOH

June 4, 2026

Contents

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Savings is a UUPS-upgradeable ERC4626 yield vault. Depositors receive `sUSDp` shares; yield accrues by inflating `totalAssets` over time at a governance-set per-second `rate`. When yield is settled (`_accrue`), the contract mints new USDp directly via `ITokenP.mint(address(this), earned)`. Share price grows because the asset pool grows while the share supply stays flat between deposits/withdrawals.

This audit covers the differential changes implemented to mitigate a donation attack vector. The vector becomes exploitable under two conditions: a prolonged period of stale accrual (no yield accrued) and an attacker controlling a disproportionate share of the total sUSDP supply.

Prior to this upgrade, the surplus from donations was indistinguishable from legitimate yield and silently flowed to existing holders.

Now, direct transfers are invisible to the vault's accounting, only real deposits and legitimate yield is included on the vault's new variable `storedAssets` which is used as the single source of truth for all accounting.

5 Audit Scope

The audit scope was limited to changes on PR 27 at commitHash 7c24bc:

```
contracts/interfaces/ISavings.sol
contracts/interfaces/ISetters.sol
contracts/parallelizer/configs/DiamondInitializer.sol
contracts/parallelizer/configs/Test.sol
contracts/parallelizer/facets/SettersGuardian.sol
contracts/parallelizer/libraries/LibSetters.sol
contracts/parallelizer/savings/Savings.sol
contracts/parallelizer/utils/Constants.sol
contracts/parallelizer/utils/Errors.sol
```

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the Parallel Savings Fix code provided by Parallel. The findings consist of 2 Low severity issues with the remainder being informational.

6.1 Operational Notes

Yield Accounting: Burn, Not Direct Transfer

Yield is minted by the Savings module, so any component that generates yield must burn it rather than transfer it directly to Savings; a direct transfer bypasses the minting flow, leaving the yield unaccounted for and subject to being swept as surplus.

6.2 Post Audit Recommendations

Pre-upgrade step: Call `setRate` to refresh the last accrual just before the upgrade, reducing stale accrual time to ~0.

Summary

Project Name	Parallel Savings Fix
Repository	parallel-parallelizer
Commit	7c24bc40d50b...
Audit Timeline	June 2nd - June 4th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	2
Informational	4
Gas Optimizations	0
Total Issues	6

Summary of Findings

[L-1] Upgrading the `Savings` contract exposes accounting to donation attack by inflating `storedAssets` via a direct donation just before the upgrade runs.	Open
[L-2] Pausing the vault does not halt yield accrual; the entire paused interval is minted at unpause	Open

[I-1] If the Savings minter role on TokenP is removed, `setRate(0)`/`setMaxRate(0)` revert because they accrue first	Open
[I-2] Operational guidance: the Savings contract must never be configured as a surplus payee	Open
[I-3] `Savings::initializeStoredAssets` lacks access control, allowing any caller to seed `storedAssets` if the upgrade is executed non-atomically	Open
[I-4] `Savings::initialize` does not set `lastUpdate` - safe-by-accident defense-in-depth gap	Open

7 Findings

7.1 Low Risk

7.1.1 Upgrading the `Savings` contract exposes accounting to donation attack by inflating `storedAssets` via a direct donation just before the upgrade runs.

Description: The upgrade is prone to being hijacked by a direct donation because it reads the raw USDP balance held by the Savings contract. If interests haven't been accrued for a prolonged period of time, this function, which forcefully sets storedAssets to the raw USDP balance held by the Savings contract, exposes the system to the donation attack.

Because the upgrade routine forcibly sets `storedAssets` equal to the contract's raw USDP balance, any discrepancy between the actual balance and the system's expected accounting is silently absorbed into protocol state. This risk is amplified when interest has not been accrued over a prolonged interval, leaving the accounting stale and the gap between real and expected balances wide open for exploitation.

Attack Scenario:

1. The preconditions mirror those of the original donation attack: an actor holds a disproportionately large share of sUSDP, and interest accrual has been stale for an extended period.
2. The upgrade transaction is submitted.
3. The attacker front-runs the upgrade, transferring a large quantity of USDP directly into the Savings contract.
4. The upgrade executes and resets storedAssets to the contract's raw USDP balance—now artificially inflated by the donation in step 3.
5. Leveraging the inflated storedAssets, the attacker redeems their sUSDP position at a corrupted exchange rate, minting a disproportionately large amount of USDP and extracting value far exceeding their legitimate share.

Recommended Mitigation: Harden `initializeStoredAssets` by enforcing a staleness bound on `lastUpdate`. Requiring `block.timestamp - lastUpdate` to stay within a strict threshold (e.g., 30 minutes or less), reverting otherwise. This caps the accrual delta available to an attacker: with a tight freshness window, there is insufficient unaccrued interest to compound against an inflated `storedAssets`, preventing the donation-driven discrepancy from being amplified over a prolonged interval.

Parallel

Cyfrin:

7.1.2 Pausing the vault does not halt yield accrual; the entire paused interval is minted at unpause

Description: `pause()` and `unpause()` both call `_accrue()`, and neither touches `rate`:

```
function pause() external restricted {
    if (paused == 1) revert AlreadyPaused();
    _accrue();          // settles to now, sets lastUpdate = block.timestamp
    paused = 1;
}

function unpause() external restricted {
    if (paused == 0) revert NotPaused();
    _accrue();          // accrues over block.timestamp - lastUpdate (the whole paused span)
    paused = 0;
}
```

`_accrue()` has no notion of the paused state — it always accrues over wall-clock time since `lastUpdate`:

```
newTotalAssets = _computeUpdatedAssets(storedAssets, block.timestamp - lastUpdate);
```

`lastUpdate` is frozen at the pause timestamp and `rate` stays non-zero, so `unpause()` mints interest for the full paused duration. Pausing only defers minting; it does not stop yield.

Impact: When governance pauses in an emergency, the protocol's token liability keeps growing for the entire paused window and is minted in full on unpause — the same result as never having paused. The pause provides no halt of emission.

Proof of Concept:

1. `rate > 0`, vault active.
2. `pause()` at `t0` (settles to `t0`, `paused = 1`).
3. 30 days pass.
4. `unpause()` at `t0 + 30d` → `_accrue` runs with `exp = 30 days` and mints 30 days of interest.

`totalAssets()` immediately after unpause equals the value it would have had if the vault had never paused for those 30 days.

Recommended Mitigation: If pausing should halt yield, drop the paused span on unpause instead of accruing it:

```
 function unpause() external restricted {
     if (paused == 0) revert NotPaused();
-    _accrue();
+    lastUpdate = uint40(block.timestamp); // skip the paused interval rather than minting it
     paused = 0;
 }
```

If deferral is intended, document that pausing does not stop emission.

7.2 Informational

7.2.1 If the Savings minter role on TokenP is removed, `setRate(0)`/`setMaxRate(0)` revert because they accrue first

Description: If the AccessManager removes the Savings contract's minter role on `TokenP`, then setting the rate to `0` will fail — because `setRate`/`setMaxRate` call `_accrue()` before mutating the rate, and `_accrue()` mints TokenP whenever yield has accrued:

```
function setRate(uint208 newRate) external onlyTrustedOrRestricted {
    if (newRate > maxRate) revert InvalidRate();
    _accrue();          // mints TokenP first
    rate = newRate;
    emit RateUpdated(newRate);
}

function _accrue() internal returns (uint256 newTotalAssets) {
    ...
    if (earned > 0) {
        ITokenP(asset()).mint(address(this), earned); // reverts: vault no longer a minter
        storedAssets = newTotalAssets;
    }
}
```

`TokenP.mint` is `restricted`, so once the role is revoked the call reverts with `AccessManagedUnauthorized`, `_accrue()` reverts, and `setRate(0)`/`setMaxRate(0)` revert with it. Governance therefore cannot zero the inflation rate after removing the role — the disabling call itself depends on the role it just removed.

Why informational:

This is a **trusted-admin** / **operational ordering** observation, not an exploitable vulnerability:

- `TokenP` has no `Pausable`, no supply cap, and no blacklist hook (verified in `parallel-tokens/contracts/tokens/TokenP/TokenP.sol`), so `mint` cannot revert organically. The only trigger is the AccessManager deliberately revoking the vault's minter role.
- Revoking the role is a deliberate governance action, and the vault cannot operate without it; the situation is self-inflicted and recoverable (re-grant the role, set rate, then revoke).
- No funds are lost; the underlying stays backed in the vault.

It is worth noting only so that an operator decommissioning the vault knows to **set** `rate = 0` **before removing the minter role**, not after.

Recommended Mitigation: It is necessary to accrue (settle outstanding yield) **before** removing the minter role: while the vault is still a minter, call `setRate(0)` so a final `_accrue()` succeeds, and only then revoke the role. Decommissioning in the reverse order traps the rate.

7.2.2 Operational guidance: the Savings contract must never be configured as a surplus payee

Description: Operational guidance, not a code defect. After the donation guard, the vault counts only `storedAssets` as backing; any USDp reaching it by a **direct transfer** is excluded from `totalAssets()` and is sweepable by `recoverSurplus`. The parallelizer's surplus distribution pays payees by direct transfer:

```
// LibSurplus._release(...)
IERC20(address(_ts.tokenP)).safeTransfer(_payee, income);
```

So if the Savings contract is set as a surplus **payee**, the released USDp lands as an ignored "donation" — it does not reach savers and is instead recoverable by governance. Saver yield is delivered only by minting at `setRate`, never by transferring USDp in.

Impact: Surplus sent to the vault is not credited to share holders, but remains rescuable by governance via `recoverSurplus`.

Recommended Mitigation: Never include the Savings contract in `updatePayees` (remove it if it ever appears in `ts.payees`); deliver saver yield only through `setRate`.

Parallel: Cyfrin:

7.2.3 `Savings::initializeStoredAssets` lacks access control, allowing any caller to seed `storedAssets` if the upgrade is executed non-atomically

Description: `Savings::initializeStoredAssets()` is a one-shot reinitializer introduced in v2 of the Savings contract to seed the `storedAssets` tracking variable with the contract's current token balance at upgrade time. The variable is central to the donation-attack fix: all subsequent yield accrual, deposit accounting, and surplus recovery depend on it being correctly initialized.

The function carries only OZ's `reinitializer(2)` guard and no role-based access control:

```
function initializeStoredAssets() external reinitializer(2) {
    storedAssets = IERC20Metadata(asset()).balanceOf(address(this));
}
```

Every other state-writing governance function in `Savings` — `pause`, `unpause`, `setRate`, `setMaxRate`, `toggleTrusted`, `recoverSurplus` — carries the `restricted` modifier. `initializeStoredAssets` is the sole exception.

Under the current deployment procedure (`upgradeToAndCall` with the initialization encoded as call data), this window is zero — upgrade and initialization occur atomically.

However, correctness relies entirely on this deployment convention being followed. A future upgrade executed as two separate transactions — due to a tooling change, governance multisig batching error, or script regression — would reopen the window.

Recommended Mitigation: Add the `restricted` modifier to `initializeStoredAssets()` so that only the authorized governance role can invoke it, regardless of whether the upgrade is executed atomically.

The modifier costs one storage read and eliminates the dependency on deployment-procedure correctness as the sole protection for this critical initialization step.

Parallel: Cyfrin:

7.2.4 `Savings::initialize` does not set `lastUpdate` - safe-by-accident defense-in-depth gap

Description: `Savings::initialize` configures the proxy storage (ERC4626 / ERC20 names, AccessManager, seed deposit) but never assigns `lastUpdate`. After deployment, `lastUpdate == 0` while `rate == 0` and `storedAssets` carries only the seed. The first state-mutating entry point triggers `_accrue`, which computes `block.timestamp - lastUpdate` (on the order of 1.7e9 seconds since the Unix epoch). The short-circuit on line 139 (`if (exp == 0 || ratePerSecond == 0) return currentBalance;`) saves the contract because `rate == 0` at boot, so no interest is computed for the multi-decade `exp`.

The safety is purely accidental: it depends on the convention that every code path that mutates `rate` calls `_accrue` first. Any future setter, migration helper, or refactor that writes to `rate` without calling `_accrue` before the first user interaction would compute interest over the entire `block.timestamp - 0` window with the just-set rate, producing astronomical mint values constrained only by the Taylor polynomial's overflow boundary.

Impact: Defense-in-depth gap that turns a single missed `_accrue` invocation in a future rate-mutating path into a catastrophic over-mint. Off-chain consumers that read `lastUpdate` as a freshness signal also see a nonsensical value (`0`) until the first interaction, which may misrepresent vault age in monitoring dashboards.

Recommended Mitigation: Set `lastUpdate` at the end of `initialize` so that the elapsed window is always bounded to "time since deployment" rather than "time since Unix epoch":

```
function initialize(...) public initializer {
    // ...existing body...
    _deposit(msg.sender, address(this), 10 ** (asset_.decimals()) / divizer, BASE_18 / divizer);
    lastUpdate = uint40(block.timestamp);
```

```
}
```

Parallel: Cyfrin: